

NLP PLATFORM SANAD

A Collaborative Platform for Researchers



Project Members

- BEZZA AMINA
- BETTAYEB MOHAMED AIMEN
- MEZIANE LINA LOUBNA
- DAHMANE YANES

Project Description:

sanad is a collaborative digital platform designed to support researchers by enabling knowledge sharing, resource management, and community interaction in a professional environment

Date

April 2026

Sommaire:

Chapter 1 — Introduction

1.1 Context and Problem Statement

1.2 Proposed Solution

1.3 Objectives

2- Problem Statement

2.1 Fragmentation in the NLP Ecosystem

2.2 Access Barrier

2.3 Trust Barrier

2.4 Continuity Barrier

3- Conceptual Positioning

3.1 Educational Dimension

3.2 Scientific Dimension

3.3 Professional Dimension

3.4 Community-driven Dimension

3.5 Core Philosophy

4- Product Vision

4.1 User-Centric Principles (Clarity, Relevance, Linguistic Inclusivity, Reliability, Evolution)

4.2 Continuous Learning Journey

5- Theoretical User Experience

5.1 Explore Phase

5.2 Understand Phase

5.3 Build Phase

5.4 Progress Phase

6- Value Proposition for Stakeholders

6.1 Students

6.2 Researchers & Educators

6.3 Companies & Institutions

7- Functional Scope

7.1 Django Application

7.1.1 User Management

7.1.2 Content Management

- Tools

- Courses
- Events
- Institutions
- Corpus (Datasets)
- Dynamic Content: Feed & News
- Opportunities
- Projects
- Topics

7.3.1 Following System

7.3.2 Approval and Moderation System

8- Techniques and Technologies Utilized

8.1 Backend and Web Infrastructure

8.2 AI and NLP Services

8.3 Data Acquisition and Processing

8.4 Frontend and User Experience

8.5 Governance and Community Features

9- Conclusion

9.1 Practical Utility

9.2 Reliability and Trustworthiness

9.3 Continuous Adaptation

9.4 Technological Integration and Impact

Abstract:

This report presents the design and implementation of a modular web platform dedicated to Natural Language Processing (NLP). In response to the fragmentation of collaborative, documentation, and analytical tools in both academic and professional environments, we propose a centralized solution that integrates semantic search, content management, real-time collaboration, advanced administration, as well as an intelligent chatbot and a web scraping module for automated data enrichment.

The architecture is based on Django 5.1 running in ASGI mode, with Celery handling asynchronous task execution. It incorporates Elasticsearch for text-based search, Qdrant for vector search, and PostgreSQL with the pgvector extension for storing structured data and embeddings. The chatbot relies on a multilingual RAG (Retrieval-Augmented Generation) pipeline that combines intent classification, request routing, hybrid retrieval, and response faithfulness verification. In parallel, the web scraping module enables the collection, filtering, and integration of external content to continuously enrich the knowledge base. The platform is containerized using Docker, while monitoring and observability are ensured through Prometheus and Grafana.

The results obtained confirm the feasibility and relevance of this integrated approach, highlighting a scalable architecture, measurable performance, and a consistent user experience. This report details the entire process, from the initial problem statement and design decisions to implementation, evaluation, and future perspectives

chapter 1— Introduction:

The digital transformation of organizations has led to a continuous growth in textual data, including administrative documents, educational content, collaborative messages, technical exchanges, and specialized knowledge bases. In this context, traditional document management approaches quickly reveal their limitations when faced with heterogeneous formats, increasing data volume, and the need for fast access to relevant information.

Recent advances in artificial intelligence—particularly large language models (LLMs), Retrieval-Augmented Generation (RAG) architectures, and vector search techniques—now offer practical opportunities to design platforms capable of understanding natural language queries, assisting users, and leveraging internal knowledge in a more intelligent and efficient way

1.1 Context and Problem Statement

The development of Natural Language Processing (NLP) technologies has significantly transformed the way organizations manage, search, and leverage content. In both academic and professional environments, teams increasingly face the challenge of information fragmentation: documents are scattered, interactions are distributed across multiple tools, and there is often no structured collective memory. This fragmentation negatively impacts productivity and limits the effective reuse of knowledge.

Recent advances in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) architectures have enabled systems to move beyond traditional keyword-based search. By combining textual indexing with vector representations, it is now possible to query knowledge bases using natural language, extract relevant entities, summarize documents, and provide context-aware recommendations.

At the same time, modern platforms require more than just search capabilities. They must also provide intelligent conversational assistance and support the automated acquisition of external knowledge. In practice, this involves integrating both an intelligent chatbot and a web scraping pipeline within the same system architecture.

However, most existing solutions remain either too generic—such as SaaS platforms with limited customization—or too fragmented, requiring the complex integration of heterogeneous tools. This leads to the following central question:

How can we design and implement a modular, containerized, and maintainable web platform that unifies content management, real-time collaboration, advanced NLP capabilities, an intelligent chatbot, and web scraping-based knowledge enrichment within a single coherent system?

Addressing this challenge involves several key technical considerations:

- selecting appropriate technologies for hybrid search (textual and vector-based);
- designing an asynchronous architecture capable of handling both resource-intensive processes (OCR, embeddings, scraping pipelines) and real-time interactions (chat, notifications, chatbot sessions);
- ensuring modularity and scalability so that new features can be integrated without disrupting the existing system

1.2 Proposed Solution

The proposed solution is a modular web application called **Sanad**. It is built on Django 5.1 as the core framework, running in ASGI mode to natively support WebSocket connections alongside traditional HTTP requests. This enables Django Channels to efficiently handle real-time bidirectional communication.

For data persistence, PostgreSQL with the pgvector extension is used to store embeddings in standard use cases. For more advanced vector search requirements, Qdrant is employed as a dedicated vector database optimized for Approximate Nearest Neighbor (ANN) search. Elasticsearch is used for advanced text indexing and retrieval, enhanced with phonetic and ICU plugins.

Redis plays a central role in the system architecture, serving as a Celery broker, result backend, application cache, real-time session manager, and storage for authentication one-time passwords (OTPs). Celery, together with Celery Beat, is responsible for executing asynchronous and computationally intensive tasks such as OCR processing (via PyTesseract), PDF processing (via PyMuPDF), embedding generation, email delivery, and scheduled maintenance. Nginx handles HTTP traffic, gzip compression, and static file delivery.

Two intelligent components form the core functional value of the platform:

- an **intelligent chatbot** based on a Retrieval-Augmented Generation (RAG) approach, capable of processing natural language queries, leveraging both internal and external data sources, and generating context-aware responses;
- an **intelligent web scraping module** designed to collect, filter, and enrich content from targeted external sources, continuously improving the platform's knowledge base.

The entire infrastructure is orchestrated using Docker Compose, ensuring reproducibility across development, testing, and deployment

environments. System monitoring is achieved through Prometheus for metrics collection and Grafana for visualization via interactive dashboards.

1.3 Objectives

The objectives of this work are structured around three complementary dimensions: functional, technical, and academic

♦ **Functional Objectives**

- Provide comprehensive user account management, including secure authentication, two-factor authentication (2FA), and session control.
- Enable the creation, organization, and monitoring of collaborative projects with dedicated discussion spaces.
- Offer a structured community forum along with a document management system incorporating an approval workflow.
- Integrate a hybrid search engine (textual and vector-based) accessible through a unified interface.
- Provide real-time notifications and direct messaging capabilities.
- Include an intelligent chatbot for conversational assistance and context-aware information retrieval.
- Integrate a web scraping module for automated content collection and enrichment

♦ **Technical Objectives**

- Design a layered architecture that is clearly structured, testable, and scalable.
- Ensure full containerization using Docker to facilitate deployment and reproducibility.
- Achieve measurable software quality through testing, static analysis, and secure coding practices.

- Ensure system observability through real-time metrics and configured alerting mechanisms.
- Decouple chatbot and web scraping services to allow independent evolution without impacting the stability of the core platform
- ♦ **Academic Objectives**
 - Demonstrate mastery of core software engineering concepts, including architectural patterns, domain-driven design, and RESTful APIs.
 - Illustrate the practical integration of NLP technologies within a production-oriented web application.
 - Demonstrate the design and integration of intelligent chatbot and web scraping components within a modular architecture.
 - Produce reusable technical and documentation resources to support future work

Chapter 2 — Solution Overview:

2.1 vue d'ensemble de l'architecture

The architecture of **Sanad** is based on a strictly layered model. This structure is not merely organizational; it reflects deliberate design choices aimed at ensuring testability, component substitutability, and clear separation of responsibilities. Each layer interacts with adjacent layers through explicit interfaces (HTTP APIs, standardized network protocols, message queues), which enhances the system's robustness in the face of both functional and technical evolution

The system is organized around five main layers: the application layer (Django running in ASGI mode), the asynchronous layer (Celery and Redis), the data layer (PostgreSQL, Qdrant, and Elasticsearch), the exposure layer (Nginx), and the monitoring layer (Prometheus and Grafana). This architecture is further enhanced by integrated intelligent modules, including:

1. an **intelligent chatbot** based on a RAG approach, designed to handle natural language queries and generate context-aware responses;
2. a **web scraping module** responsible for the collection, structuring, and automatic enrichment of external data to support the knowledge base

This design ensures that computationally intensive processes—such as embedding generation, OCR, or scraping pipelines—do not impact the main application response cycle. In addition, textual search and vector search remain decoupled, allowing the underlying engines to evolve independently, as long as interface contracts are maintained

2.2 Application Layer — Django & ASGI

Django 5.1 forms the core of the application layer. The choice of this framework is based on its architectural maturity, the robustness of its ORM, its proven authentication system, its automatically generated administration interface, and its rich ecosystem of third-party libraries. In this version, Django also benefits from enhanced integration with ASGI mode, making it particularly suitable for hybrid use cases combining traditional HTTP requests and real-time communication.

The ASGI entry point defined in `asgi.py` aggregates multiple WebSocket routers dedicated to notifications, direct messaging, project interactions, chatrooms, and scraping-related event tracking. This configuration allows a single Daphne server to handle both HTTP requests and persistent WebSocket connections, reducing deployment complexity while improving system responsiveness.

Django REST Framework provides the RESTful API layer. Views organized through ViewSets and Serializers enable rapid development of full CRUD APIs while offering automatic validation, a browsable API useful during development, and seamless integration with Django's authentication system. Internationalization is handled through Django's native `i18n` mechanism, with structured locale-based translation files. On the

frontend side, HTMX is used to introduce interactivity without relying on a heavy JavaScript framework, following a lightweight server-driven rendering approach

2.3 Asynchronous Layer — Celery & Redis

Tasks that exceed a few seconds or require deferred execution are delegated to the Celery asynchronous layer. This mechanism is essential to maintain interface responsiveness, particularly during document indexing, embedding generation, scraping pipelines, or chatbot-related processing tasks.

Redis acts as both a message broker and a result backend. Celery workers consume tasks stored in Redis, execute them, and store results according to the configured backend strategy. This architecture improves fault tolerance, enables horizontal scaling of workers, and prevents blocking the main Django execution thread. Celery Beat handles scheduled tasks such as data cleanup, periodic index synchronization, and automated task execution.

The presence of multiple queues and workers reflects a clear separation of concerns. Django business logic tasks, document processing workflows, and NLP/chatbot-related operations do not necessarily share the same execution pipeline, reducing contention and improving overall system stability

2.4 Data Layer — PostgreSQL, Qdrant & Elasticsearch

The data layer is intentionally heterogeneous to address the diversity of the platform's functional requirements.

PostgreSQL with the pgvector extension serves as the primary relational database. All business entities—such as users, projects, messages, and resources—are persisted through Django's ORM. The addition of pgvector

enables the storage and querying of vector embeddings directly within the relational database, which is suitable for moderate-scale similarity search use cases.

Elasticsearch is dedicated to advanced full-text search. It efficiently indexes application and document content using precise mappings and provides relevance-based lexical search capabilities. This component is particularly useful for keyword search and for enriching user-facing search results.

Qdrant is used as a specialized vector database for large-scale semantic search. Its native support for collections, hybrid filtering (vectors + metadata), and high-performance Approximate Nearest Neighbor (ANN) search makes it a key component for retrieval-augmented scenarios, especially those used by the chatbot. The combination of PostgreSQL, Elasticsearch, and Qdrant enables a hybrid search strategy that merges lexical relevance with semantic similarity

2.5 Exposure Layer — Nginx & WebSockets

Nginx acts as the single entry point to the platform. As a reverse proxy, it forwards HTTP requests to Django or to the Daphne server depending on the deployment configuration, while also handling WebSocket connections. This centralization simplifies traffic management, routing, and access security.

Gzip compression is enabled for textual responses, reducing bandwidth usage. Nginx also serves static and media files directly from shared volumes, avoiding unnecessary I/O load on the application layer. In production environments, the configuration can include standard TLS settings and HTTP-to-HTTPS redirection to ensure secure communication.

WebSocket support is essential for real-time modules such as messaging, notifications, chatrooms, and scraping event tracking. This exposure layer

therefore plays a critical role in maintaining system coherence and responsiveness.

2.6 Monitoring Layer — Prometheus & Grafana

Observability is a core requirement of the platform. Key system metrics are exposed to Prometheus from critical components, including Django, Celery, Redis, PostgreSQL, Nginx, and related services. This instrumentation enables continuous monitoring of system health and early detection of performance degradation.

Grafana is used to visualize these metrics through automatically provisioned dashboards. Key indicators include HTTP request rates, response latency, Celery queue lengths, memory usage, WebSocket connection states, and potential bottlenecks in data processing pipelines.

Alerting rules defined in `alerts.yml` identify critical situations such as CPU saturation, excessively long task queues, or degraded search performance. This monitoring setup enhances proactive system supervision and significantly improves maintainability.

2.7 Functional Modules

The platform is structured into independent Django modules, each containing its own models, views, serializers, WebSocket routes, and tests. This modular organization improves maintainability, enables parallel development, and reduces cross-module dependencies.

The **accounts** module handles authentication, user management, and security mechanisms. The **projects** module structures collaboration around project spaces, while **project_chatroom** and **forum** enable real-time communication via WebSockets. The **notifications** module manages instant alerts, and **direct_messages** handles private messaging.

The **resources** module represents the core document management system. It handles file upload, content extraction, and indexing, leveraging

specialized libraries for PDF, DOCX, images, and other document formats. The **search** module provides unified access to all indexed sources through a centralized search interface.

The **chatbot** module implements the platform's conversational assistant. Based on the application code, it communicates with a separate FastAPI service, maintains conversation sessions, and can reuse contextual information from specific business content to generate contextualized responses. This makes it an intelligent retrieval and generation component, complementary to traditional document search.

The **scraping** module completes the system by handling external data acquisition. It provides dedicated views for launching, monitoring, and restarting scraping tasks, enabling continuous enrichment of the platform's datasets and improving both search and chatbot knowledge sources

2.8 Deployment Pipeline — Docker & Services

Deployment is based on Docker Compose to ensure environment reproducibility and service isolation. Each major component of the platform is encapsulated in a separate container, simplifying startup, monitoring, and scalability.

The orchestration includes PostgreSQL with pgvector, Redis, Qdrant, Elasticsearch, Django, the FastAPI chatbot service, Celery workers, and Nginx. This separation of services clearly distinguishes responsibilities such as data persistence, lexical search, vector search, asynchronous processing, web exposure, and conversational processing.

This approach offers several advantages. It reduces discrepancies between development and production environments, simplifies maintenance operations, and allows resource allocation to be adjusted based on each service's actual workload. In this architecture, Docker is

not merely a packaging tool but a structural component of the software design itself

Chapter 3 — Practical Implementation:

3.1 Development Environment

Setting up a reproducible development environment is a fundamental step in building a large-scale software platform. In a system that integrates multiple interconnected services, asynchronous processing, a relational database, a vector database, a full-text search engine, a chatbot module, and a web scraping module, manual installation on each development machine would inevitably lead to configuration inconsistencies, version incompatibilities, and debugging difficulties.

To address this constraint, the platform relies on **Docker Compose** as a local orchestration tool. The entire infrastructure is defined in a single configuration file, which explicitly specifies services, environment variables, persistent volumes, internal networks, and inter-service dependencies. This allows the environment to be launched in a standardized way, without requiring machine-specific installation procedures.

The Django application Dockerfile is based on a lightweight Python image in order to reduce the attack surface and minimize the final image size. System dependencies are installed only when necessary, and installation caches are cleaned to optimize build performance. This approach improves security, maintainability, and reproducibility.

The development ecosystem also includes a dedicated FastAPI service for the chatbot, Celery workers for background processing, a PostgreSQL service with vector extensions, Redis for queues and caching, Qdrant for semantic search, Elasticsearch for text search, and Nginx for network exposure. This structure reflects a distributed architecture while remaining easy to run locally thanks to Docker

3.2 Docker Infrastructure Setup

The startup sequence of services is a critical aspect of this architecture. Some components, such as Elasticsearch or PostgreSQL, require initialization time before becoming operational, while Celery workers must wait for Redis and storage services to be fully available. Without synchronization mechanisms, simultaneous container startup would lead to transient errors and hard-to-reproduce failures.

This dependency management is handled using a combination of `depends_on` and Docker healthchecks. Each service exposes a health verification mechanism suited to its nature: `pg_isready` for PostgreSQL, `redis-cli ping` for Redis, and HTTP health endpoints (via curl) for Elasticsearch and web services. This approach ensures controlled startup order without relying on fragile sleep-based scripts or arbitrary delays.

The PostgreSQL initialization script enables required extensions at volume creation time, particularly `pgvector`. This step is essential to ensure that the relational database supports both classical business data and vector representations used by the NLP layer. It also guarantees clean reinitialization without structural loss.

Persistent volumes ensure data retention across container restarts. They are used for PostgreSQL, Redis, Qdrant, Elasticsearch, static files, media files, and application caches. This persistence is particularly important for indexed documents, chatbot conversation history, and datasets collected through scraping. The FastAPI chatbot service also uses a dedicated cache for models and temporary resources to reduce startup time and avoid repeated downloads.

3.3 Core Module Development

Accounts Module — Secure Authentication

The accounts module implements user authentication and account management. The integration of `django-allauth` enables an email-based authentication flow while maintaining a high degree of customization. This flow is enhanced with additional security mechanisms, including a one-time password (OTP) verification step after credential submission.

The OTP code is generated server-side, stored temporarily in Redis, and assigned a limited validity period. This ensures automatic expiration after a few minutes, reducing the risk of reuse or abuse. Using Redis as a session backend also allows user state sharing across multiple application instances, facilitating horizontal scaling

Search Module — Hybrid Search

The search module is one of the core components of the platform. It unifies search across business entities and document resources by combining Elasticsearch for textual indexing and Qdrant for semantic vector search. This dual approach overcomes the limitations of purely lexical search.

Indexed entities include users, projects, resources, events, and institutions. Each category has its own indexing schema tailored to its structure and usage. Text search provides efficient keyword-based retrieval, while vector search captures semantic similarity between contents.

This hybrid search strategy is particularly valuable for the chatbot, which relies on the same data sources to retrieve relevant content and generate contextualized responses

Notifications Module — Real-Time Delivery

The notifications module is built on Django Channels to provide real-time event delivery. Each connected user maintains a persistent WebSocket connection, allowing instant delivery of alerts without page refresh. This

capability is essential for messages, invitations, discussion replies, and collaboration events.

The system uses per-user channel groups to target notifications precisely. If a user is offline at the time of an event, the notification is stored in the database and delivered later. This ensures continuity of information and improves user experience

Chatbot Module — Intelligent Conversational Assistant

The chatbot module is one of the most visible components of the platform. It goes beyond a traditional chat interface and is designed as a retrieval-augmented generation system. It stores conversation history, exchanged messages, and user feedback, ensuring continuity and contextual awareness across sessions.

According to the implementation, the Django chatbot acts as an integration layer with a separate FastAPI service. This service handles conversation requests, streaming responses, quick queries, advanced search, and explanation-based interactions over internal platform content. This separation isolates heavy NLP logic from the main web application, improving scalability and performance control.

The chatbot can also leverage contextual information when launched from a specific resource, project, or content page. In this case, it becomes a context-aware assistant capable of answering based on the current object rather than generating generic responses. This significantly enhances the platform's educational and functional value

Scraping Module — External Data Acquisition

The scraping module is responsible for enriching the platform with structured external data. It is not a simple page collector, but a full pipeline for collection, normalization, validation, and enrichment of external sources. The module provides dedicated interfaces for starting

scraping jobs, monitoring execution, restarting runs, testing sources, and managing custom scraping configurations.

Scraping configuration is centralized, allowing fine-grained control over timing, retry policies, deduplication thresholds, pagination limits, maximum document sizes, and enrichment rules. This centralization improves system clarity and avoids scattering technical constants across multiple files.

Scraping plays a key role in the overall system. On one hand, it enriches Elasticsearch indices and Qdrant vector stores. On the other hand, it provides fresh and contextual data that can be used by the chatbot. Together, scraping and the chatbot form a coherent pipeline: one collects knowledge, the other transforms it into intelligent responses

3.4 NLP Integration and Advanced Search

The NLP integration represents the most innovative part of the platform. It is based on a pipeline combining embedding generation, hybrid retrieval, and contextual response generation. The goal is not only to retrieve documents, but to produce intelligent, intent-aware answers.

Embedding generation is performed using sentence-transformer models that convert text into dense vector representations. Since this process is computationally expensive, it is executed asynchronously via Celery to maintain interface responsiveness. Generated vectors are stored in Qdrant along with metadata for filtering and source identification.

The document processing pipeline integrates multiple tools depending on file formats. Text from PDFs is extracted using dedicated parsers, scanned images and documents are processed using OCR when needed, and office files are handled by specialized libraries. The content is then normalized before indexing. This pipeline feeds both lexical and semantic search systems.

The chatbot architecture follows a RAG paradigm. When a user submits a query, the system combines Elasticsearch (keyword search), Qdrant (semantic search), and, when necessary, external web sources via the FastAPI service. The retrieved results are merged and used as context for response generation, producing more accurate, relevant, and explainable answers.

Scraping completes this ecosystem by continuously feeding external data into the system, while the chatbot leverages this enriched corpus to generate high-quality responses. This creates a full pipeline from data acquisition to intelligent information delivery

3.5 Real-Time System and WebSockets

The platform's real-time architecture is built on Django Channels with Redis as the channel layer. The `asgi.py` file aggregates multiple WebSocket routers, enabling notifications, direct messaging, project-related communication, chatrooms, and scraping monitoring streams within a single application. This design unifies communication channels while maintaining clear separation of responsibilities.

WebSocket consumers serve different purposes. Some are limited to event broadcasting, while others handle more complex business logic such as access control, message persistence, history management, or typing indicators. This architecture improves responsiveness and enables a smooth user experience.

WebSocket security is treated with the same rigor as HTTP security. Authentication is performed at connection time, and access rights are validated before allowing subscription to channels. This ensures confidentiality, particularly in collaborative spaces and chatbot-related interactions

3.6 Software Quality and Testing

Software quality was integrated from the early stages of the project design. Tests are structured to cover core business logic, real-time components, chatbot functionalities, and scraping mechanisms. This strategy reduces regressions and secures future evolution.

Unit tests validate isolated functions, validators, and utilities. Integration tests use a real database and Django's test client to validate views, permissions, and API endpoints. End-to-end tests simulate real user workflows, including authentication, search, collaboration features, and chatbot interactions.

Static analysis and code quality tools reinforce this approach. Linting, security analysis, and type checking help detect issues early in the development cycle. This discipline is essential in a system combining Django, FastAPI, Celery, WebSockets, vector search, and scraping, where complexity can

3.7 Security System — Authentication and Access Control

The security system represents a fundamental pillar of the Sanad platform. Email-based authentication via `django-allauth` provides a secure and customizable login flow, enhanced by an additional layer of multi-factor authentication (2FA). After credential submission, a One-Time Password (OTP) is generated server-side and temporarily stored in Redis with a limited validity period of a few minutes. This mechanism ensures automatic expiration and significantly reduces the risk of reuse or abuse.

Redis plays a central role as a session backend, enabling user state sharing across multiple application instances and supporting horizontal scalability. Beyond authentication, the system implements Role-Based Access Control (RBAC), applied at the level of views, API endpoints, and WebSocket channels. Each protected endpoint validates user permissions before granting access to resources, ensuring a clear separation between public, authenticated, and administrative users. WebSocket security is

handled with the same rigor as HTTP requests, including authentication at connection time and authorization checks before subscription to channels.

3.8 Notification Module and Following System

The notification module forms the real-time communication backbone of collaboration within Sanad. Built on Django Channels with Redis as the channel layer, it maintains persistent WebSocket connections for each connected user, enabling instant delivery of alerts without page reloads. The system uses per-user channel groups to precisely target notifications. When a user is offline, notifications are persisted in the database and delivered upon reconnection, ensuring continuity and a consistent user experience.

The following system is tightly integrated with this mechanism. Users can follow other users, projects, resources, or thematic categories. Every relevant action—such as content creation, project updates, event publication, or discussion replies—automatically generates notifications delivered to followers via WebSocket channels. This real-time event stream keeps users informed without cognitive overload, a key factor in maintaining an active and engaged community. Notifications can also be filtered and customized according to user preferences, allowing each user to control their information flow.

3.9 Administration Panel — Centralized Management and Monitoring

Sanad provides a comprehensive administration panel, leveraging one of Django's most powerful features: its automatically generated admin interface. This tool allows administrators to manage all business entities—users, projects, resources, events, institutions—without

requiring custom development. The interface includes advanced filtering, bulk actions, and inline editing, significantly accelerating operational tasks.

Beyond basic CRUD operations, the administration panel exposes critical monitoring capabilities. Administrators can view ongoing indexing processes, monitor Celery task queues, inspect error logs, and access reports on system activity. This centralized access to system health reduces troubleshooting time and significantly improves maintainability. Access to the admin panel is strictly controlled through RBAC, ensuring that only authorized administrators can modify sensitive data or access critical system information.

3.10 Approval Workflow and Moderation System

The approval and moderation system ensures the quality and integrity of content shared on the platform. Certain types of content—such as resources, events, institutions, and articles—can be submitted through a structured approval workflow. Upon creation, content is placed in a “pending approval” state and remains visible only to its creator. Moderators are notified and can review the content before approving or rejecting it.

This workflow is implemented using a status field combined with fine-grained permissions. Moderators have the ability to view, edit, and approve content, while regular users can only access their own pending submissions. In case of rejection, a reason can be recorded and communicated back to the creator through the notification system, establishing a constructive feedback loop. This mechanism is essential for maintaining a trustworthy environment and ensuring consistent content quality, particularly for academic resources and institutional data where accuracy is critical.

grow rapidly without continuous quality control

3.11 User Profile and Personalization

The profile module represents each user's digital identity within the Sanad platform. It allows users to personalize their presence by adding a profile picture, biography, institutional affiliations, social links (such as LinkedIn, Twitter, etc.), and areas of expertise. This information enriches the social context of the platform and facilitates the discovery of collaborators with similar interests.

The profile also includes a personal analytics system, displaying metrics such as the number of created projects, contributions, published resources, and overall community engagement. Notification preferences, language settings, and privacy controls are centrally managed within the profile settings, allowing users to fully control their visibility and experience. A badge or accreditation system may be introduced in future versions to highlight outstanding contributions

3.12 Opportunities Module

The opportunities module centralizes announcements related to scholarships, internships, academic positions, and research calls relevant to the academic community. These opportunities may originate from internal sources (created by administrators or partner institutions) or external sources (collected via web scraping). Each opportunity includes a detailed description, eligibility criteria, application deadlines, and links to submission portals.

Users can subscribe to specific opportunity categories and receive notifications when new listings match their interests. Opportunities can also be linked to specific projects or institutions, strengthening the platform's internal connectivity and improving access to critical academic and professional information

3.13 Dynamic Feed System (News & Activity Feed)

The dynamic feed system provides a personalized stream of news, updates, and events based on each user's interests and follow relationships. Unlike targeted notifications, the feed offers a global overview of community activity, including new publications, institutional updates, project summaries, and ongoing discussions. This mechanism keeps users informed while promoting organic content discovery.

The feed incorporates a relevance-based ranking system that takes into account user engagement history, followed entities, and past interactions. Administrators can publish official news articles and important announcements that are pinned at the top of the feed. This component plays a key role in maintaining a dynamic community and fostering a strong sense of belonging among users.

3.14 Forum and Community Discussions

The Sanad forum provides a structured discussion space where users can ask questions, share experiences, and collaborate on predefined topics or categories. Each discussion is organized into threaded conversations with a voting system (like/dislike) and filtering by popularity or recency. Moderators can pin important discussions, mark answers as solutions, and ensure overall content quality.

The forum is tightly integrated with the WebSocket-based notification system, ensuring that users receive real-time alerts when their questions are answered or when they are mentioned in discussions. Forum content is also indexed in Elasticsearch, making it searchable through the hybrid search engine. This combination of synchronous interaction and asynchronous search transforms the forum into a valuable collective knowledge base.

3.15 Tools and Integrated Utilities Module

The tools module groups practical utilities designed to enhance user productivity, including format converters, text extractors, bibliographic

citation generators, data visualization tools, and other useful scripts. Each tool may be implemented either as a microservice or as an asynchronous Celery task, depending on its computational complexity.

Tools are integrated into the approval workflow, allowing advanced users or contributors to propose and submit new utilities for administrative validation. This community-driven approach gradually expands the ecosystem of tools without overloading the core system. Popular or critical tools can be promoted through the administration panel and highlighted for end users

3.16 Corpus and Dataset Management

The Corpus module forms the foundation of collective knowledge within Sanad. It enables the creation, organization, and distribution of structured datasets intended for research and model training. Each corpus is described through detailed metadata, including thematic domain, language, size, format, license, and data provenance.

Corpora can be uploaded manually or automatically acquired through the scraping module for continuous enrichment. Each dataset is versioned and can only be marked as "approved for publication" after moderation validation. Usage and download statistics are tracked to identify the most valuable resources. This centralization transforms Sanad into a reusable data repository, accelerating research workflows and reducing fragmentation of academic datasets

3.17 Courses Module

The Courses module enables the creation and sharing of structured educational content. Each course is organized into modules, lessons, and associated resources (documents, videos, quizzes). Instructors can define prerequisites, learning objectives, and a clear pedagogical pathway. Users can enroll in courses of interest and receive notifications for new lessons.

A progress tracking system monitors learner advancement and generates completion certificates. Courses can be linked to practical projects, allowing learners to immediately apply their knowledge. This module transforms Sanad into a collaborative learning platform, fostering skill development and continuous professional growth within the community

3.18 Projects Module

The Projects module represents the core of collaboration within Sanad. Each project defines its participants, objectives, progress status, and associated resources. Projects are organized into assignable tasks, internal discussions, and measurable milestones. A role and permission system clearly defines who can contribute, approve, or modify project elements.

Projects include an internal activity timeline (project feed), shared versioned files, and collaborative documentation. The chatbot can be activated in a project context, acting as an intelligent assistant capable of answering questions based on project-specific data and objectives. Public or semi-public projects can be discovered through search, enabling emerging partnerships and cross-team collaboration

3.19 Events Module

The Events module centralizes the publication and management of conferences, seminars, webinars, and workshops relevant to the community. Each event includes a description, date, location (or virtual participation link), and registration form. Events can be linked to projects or institutions, enriching the academic calendar context.

Users can register for events, express interest, or receive reminders. Moderators approve events before publication. After an event, shared resources (slides, recordings, reports) are archived and indexed for future access. This module transforms Sanad into a community

engagement hub, increasing visibility and participation in academic activities

3.20 Institutions Module

The Institutions module documents universities, laboratories, research centers, and partner organizations using Sanad. Each institutional profile includes a general overview, contact persons, areas of expertise, and associated projects or resources. Institutions can publish opportunities, events, or research corpora directly linked to their profile.

A verification system marks authenticated institutions with a trust badge, preventing impersonation and reinforcing academic credibility. Institutional profiles act as aggregation hubs: users can access all related activities, publications, opportunities, and collaborators in a single place. This fosters B2B and B2A collaboration and positions Sanad as a reliable academic intermediation platform

3.21 Integrated Tools Module

The Tools module aggregates practical utilities developed by the community or provided natively, such as format converters, text extractors, citation generators, data visualizers, statistical calculators, and other useful scripts. Each tool exposes a simple interface (HTML form or REST API) and can be executed asynchronously via Celery when computationally intensive.

Tools are catalogued, versioned, and subject to approval. Advanced users may submit new tools through a structured evaluation process. A rating and review system allows the community to highlight the most useful tools. This approach gradually transforms Sanad into a microservices-oriented ecosystem without overloading the main interface, while maintaining coherence and quality.

Chapter 4 — Conclusions and Perspectives:

4.1 Project Assessment

This project has demonstrated that it is entirely feasible to design and implement a modular web platform integrating advanced natural language processing capabilities, web scraping, and conversational assistance within reasonable time constraints, provided that a coherent architecture and rigorous technical discipline are adopted from the outset.

The chosen layered architecture proved effective in practice. The separation between the Django application layer, the Celery asynchronous layer, the text and vector search layer, and the specialized chatbot and scraping modules enabled independent evolution of system components. The integration of Qdrant for semantic search did not replace Elasticsearch, but rather complemented it, enabling a richer hybrid search experience better suited to natural language queries.

Containerization using Docker played a key role in ensuring reproducibility. Docker Compose made it possible to deploy the entire infrastructure consistently on any machine with Docker installed, significantly reducing environment-related issues. Proper handling of service dependencies and health checks was essential for stabilizing system startup.

The chatbot represents the most visible contribution from a user perspective. It is based on a RAG architecture, maintains conversation history, and can leverage contextual information from platform content to generate more relevant responses. In parallel, the web scraping module continuously enriches the system with structured external data, feeding both the indexing pipeline and the chatbot's knowledge base. Together, they form a coherent pipeline from data acquisition to intelligent information delivery.

4.2 Challenges Encountered

The development of this project was not without difficulties. The first major challenge concerned Docker infrastructure setup, particularly

service synchronization and correct handling of inter-service dependencies. In a distributed architecture involving PostgreSQL, Redis, Qdrant, Elasticsearch, Django, FastAPI, Celery, and Nginx, startup order is a critical aspect that requires precise configuration.

Another challenge involved the performance of NLP processing tasks, especially embedding generation and indexing. On limited hardware resources, these operations can become computationally expensive, particularly when applied to large document collections. Celery helped isolate these workloads from the main HTTP cycle, but latency remains an important factor as the dataset grows.

The chatbot also introduced specific challenges, especially in managing sessions, conversational context, and integration with platform content. The goal was not only to generate responses but to produce context-aware answers, which required careful coordination between the Django interface, the FastAPI service, and the stored data.

The web scraping module required particular attention in terms of robustness. The diversity of sources, page structures, update frequencies, and HTML layouts required a flexible data collection strategy rather than a simple page downloader. Data normalization, deduplication, and network error handling were key technical concerns.

Finally, multilingual support for search and document processing required several adjustments, particularly to improve indexing quality and search relevance for Arabic, French, and English content. Optimizing analyzers and language models required multiple iterations before achieving satisfactory results

4.3 Future Work and Perspectives

The implemented platform provides a solid foundation, but several improvements can be considered in the short and medium term to further enhance its capabilities and robustness.

Short-Term Functional Improvements

A natural evolution would be the integration of an external or self-hosted large language model (LLM) to enhance the chatbot's capabilities. The existing RAG architecture already provides a strong foundation, as it includes document retrieval and contextualization mechanisms.

An automatic summarization pipeline could also be introduced to generate concise summaries of long documents, improving accessibility to large content. Similarly, sentiment analysis or topic clustering modules could be added for forum discussions and collaborative exchanges.

The web scraping module could be extended to support additional data sources, more precise extraction rules, and improved automatic content classification. This would further enhance the quality of the dataset used for both search and chatbot responses.

Finally, a dedicated mobile application consuming the existing APIs would significantly improve accessibility and user experience on mobile devices.

Medium-Term Architectural Improvements

From an architectural perspective, horizontal scalability is already partially addressed through Celery workers, but the system still relies on a central Django core. A more fine-grained microservices decomposition could be considered, particularly for the NLP, chatbot, and scraping components.

A migration toward Kubernetes could also be envisioned to improve production orchestration, autoscaling, and system resilience. Introducing a message broker such as RabbitMQ or Kafka would further decouple system components and improve asynchronous workload distribution.

A semantic caching layer for repeated NLP queries could significantly reduce computation costs and improve chatbot response times.

Additionally, incremental fine-tuning of embedding models on platform-specific data could improve retrieval relevance over time

Production-Level Enhancements

For large-scale deployment, several additional improvements would be necessary: secure secrets management using a dedicated vault system, high availability for search engines, automated backup strategies for PostgreSQL and Qdrant, advanced monitoring, and a full CI/CD pipeline including testing, staged deployment, and rollback mechanisms

Final Remarks

Overall, this project demonstrates that a university-level application can evolve far beyond a simple document management system. The combination of a chatbot, web scraping, hybrid search, and a containerized infrastructure provides a strong foundation for future development toward a truly intelligent, collaborative, and scalable platform

4.4 General Conclusion

This report has demonstrated how a well-designed modular web architecture—combining Django, Docker, hybrid search technologies, and natural language processing capabilities—can transform an academic collaboration platform into a truly intelligent system. The addition of an NLP layer, including a conversational chatbot and an intelligent web scraping component, is not merely a functional enhancement; it represents a qualitative shift in how users interact with content and knowledge. The system moves from traditional keyword-based search toward semantic querying capable of understanding user intent. This marks a paradigm shift that finally leverages reasoning and summarization capabilities to better address real user needs.

The architecture is characterized by deliberate pragmatism. Rather than pursuing theoretical perfection, the design focuses on operational robustness: each technical component addresses a clearly identified functional requirement, and every added service provides measurable and justified value. The chatbot generates contextualized responses based on the full available content, while the scraping module continuously enriches the dataset, feeding both the search indices and conversational capabilities. This architectural coherence—where each component strengthens the others—illustrates how to build systems that are both sophisticated and maintainable. This “just enough engineering” philosophy is what distinguishes a truly deployable and living system from a purely academic prototype.

The **Sanad platform** demonstrates that by combining a mature web framework such as Django with specialized open-source tools—Elasticsearch for text search, Qdrant for vector search, Celery for asynchronous processing, and Docker for reproducibility—a reasonably sized team can build a production-grade technical solution. NLP capabilities, once reserved for large organizations with massive infrastructures and budgets, are now accessible to anyone who understands distributed system design and applies disciplined engineering practices. This represents a form of technological democratization: the main barrier is no longer infrastructure or tools, but architectural expertise and technical rigor.

The perspectives opened by this work are broad and promising. The integration of more powerful language models to enhance the chatbot, the extension of RAG architectures to other specialized domains, the expansion of scraping to more diverse and structured sources, and the scaling toward multi-million document corpora all represent meaningful research and development directions. From an infrastructure standpoint, migration to Kubernetes, the introduction of an event bus for stronger decoupling, or the implementation of a semantic caching layer to optimize

repeated NLP queries would further improve scalability and system resilience.

In conclusion, this work illustrates how modern web technologies, when coherently orchestrated around a clear vision, can significantly improve user experience and access to knowledge. The **Sanad platform** and its underlying architecture provide a solid foundation for future exploration of artificial intelligence and natural language processing in academic and collaborative environments. The path is open, and the tools are already available.

